

Note de cadrage — App de jam génératif (guitare → drone + rythme)

Statut : v0 — idée en cours de cadrage

Contexte

Projet perso, exploration MIR (Music Information Retrieval) temps réel. Objectif : une appli qui écoute la guitare en train d'être jouée (mélodie ou accords, indifféremment) et génère en live un accompagnement génératif — un drone dark synth accordé sur la tonalité détectée, et une couche rythmique kick/percu réactive au jeu. Contrainte non négociable : **tout tourne en local**, aucun traitement cloud.

Concept en une phrase

Écoute → détecte la tonalité et l'énergie de jeu en continu → génère un drone accordé + un rythme qui répond à la densité de jeu, avec un peu de diversité via des grilles rythmiques.

Fonctionnalités MVP (v0)

1. **Capture audio locale** (entrée micro/interface son)
2. **Détection de tonalité en continu** — chroma/CQT + Krumhansl-Schmuckler (ou `KeyExtractor` d'Essentia), avec **lissage temporel** obligatoire (fenêtre glissante ou hystérésis) pour éviter que la tonalité "flicke" à chaque frame
3. **Détection d'attaques (onset detection)** via aubio, pour moduler l'intensité/densité du rythme généré
4. **Drone synth** accordé dynamiquement sur la tonalité détectée
5. **Rythme kick/percu génératif**, densité pilotée par l'énergie de jeu détectée, avec **rythmes euclidiens** comme moteur de variété
6. Synthèse audio temps réel, faible latence

Hors scope v0 (pistes pour plus tard)

- Beat tracking causal (tempo suivi en vrai temps réel — lib **BTrack**)
- Suivi mélodique note à note / harmonisation live (nécessiterait pYIN dédié + détection mono/poly)
- Transcription d'accords fine (Basic Pitch)
- Interface graphique évoluée / contrôles utilisateur

Choix déjà tranchés

- **Un seul pipeline chroma** gère accords et notes seules — pas besoin de détecter "dans quel mode je joue", le chroma absorbe naturellement les deux cas
- **Onset detection plutôt que beat tracking** pour le v0 — plus robuste, résultat déjà organique, moins de risque de développement
- **Rythmes euclidiens** comme générateur de variété, modulés par la densité de jeu
- **Lissage temporel** non négociable sur la détection de tonalité

Architecture technique proposée

Brique	Techno	Rôle
Analyse	Python — aubio (onset + pitch), librosa/essentia (chroma + key)	L'intelligence
Synthèse	SuperCollider	Le son — drones et percus génératifs, faible latence
Liaison	OSC	Découplage total entre les

Brique	Techno	Rôle
		deux process

Alternatives envisagées : tout en C++/JUICE (binaire unique, plus lourd à développer) ou Web Audio API + Meyda.js (portable, mais écosystème MIR moins mature que Python).

Risques / points de vigilance

- Latence cumulée analyse → synthèse à calibrer finement (cible probable < 20-30 ms)
- Bruit de médiateur / bruit ambiant à filtrer en amont de l'onset detection
- Seuils de confiance sur la stabilité tonale à calibrer empiriquement, en situation de jeu réel
- Overhead de dev du setup cross-process (Python + SuperCollider) — prévoir un mode "mock" pour tester la synthèse indépendamment de l'analyse dès le début

Roadmap suggérée

1. **Preuve de concept analyse seule** — pas de son généré, juste visualisation live tonalité + onsets (terminal ou petit affichage)

2. **Drone seul** — accordé sur la tonalité détectée, sans rythme
3. **Couche rythmique** — kick/percu pilotée par onsets + grille euclidienne
4. **Réglages fins** — lissage, latence, calibration des seuils, éventuelle petite UI de contrôle (mix drone/rythme, choix de gammes de rythmes euclidiens)

Prochaine étape immédiate

Monter le prototype de phase 1 en pur Python (aubio + librosa) pour valider la fiabilité de la détection sur guitare réelle, avant d'investir dans la partie synthèse.